RAG Pipeline Design and Vector Databases

Scaler School of Technology | ML System Design and LLMOps | Week 4 — Session 1/2 | Post-Lecture Review & Practice

Key Takeaways

RAG = retrieval + generation, not just a bigger model

Retrieval-Augmented Generation injects real, sourced knowledge at inference time. It solves the knowledge cutoff and hallucination problems without the cost of full fine-tuning. Fine-tuning teaches behaviour; RAG teaches what to know.

Chunking is the hidden variable that determines retrieval quality

No vector database or embedding model can recover from bad chunking. The strategy (fixed, sentence, recursive, semantic) and chunk size (rule of thumb: 512 tokens, 50 overlap) must be tuned per dataset. Start with recursive chunking; validate with context recall scores.

Hybrid search beats pure dense retrieval in production

Dense search misses exact keyword matches (product IDs, version numbers, names). Sparse BM25 misses semantic paraphrases. Hybrid search with RRF merging covers both failure modes. Default to hybrid for any production RAG system.

Evaluation must cover retrieval AND generation

High faithfulness with low context recall means your LLM is honest but your retrieval is broken. High context recall with low faithfulness means you retrieve the right docs but the LLM ignores them. Measure all four RAGAS metrics.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. Your RAG system uses `text-embedding-ada-002` to build the index. A team member updates the query path to use `text-embedding-3-small` because it is cheaper. No errors are thrown. What happens to retrieval quality and why?

Q2. You are comparing Chroma and Qdrant for a production RAG system that handles 50M documents and requires filtering on 10 metadata fields per query. Which do you choose and why?

Q3. A RAG system returns faithfulness = 0.97 and context recall = 0.41. What does this mean in plain English? What part of the pipeline would you fix?

Q4. What is Reciprocal Rank Fusion (RRF) and why is it used to combine dense and sparse retrieval results rather than simply averaging similarity scores?

Coding / Design Exercises

Work through these independently or in pairs. No starter code — derive the approach from the lecture.

Exercise 1: Chunking Strategy for a Legal Document Corpus

You are building a RAG system over a corpus of 10,000 legal contracts (PDFs, 5–80 pages, varied structure: parties, recitals, definitions, clauses, schedules). Users ask questions like "What is the termination clause in the Smith–Acme contract?" and "Which contracts have a force majeure clause?"

Design your chunking strategy. Answer concretely: (a) Which strategy do you choose (fixed, sentence, recursive, semantic) and why? (b) What chunk size and overlap do you start with? (c) How does the highly variable document length and structure affect your choice? (d) What metadata would you store alongside each chunk (e.g., document id, page number, clause type) and how does that enable better filtering?

Exercise 2: Diagnosing a Hallucinating RAG System

A deployed RAG system answers customer support questions over a product manual. It answers correctly 80% of the time, but when it is wrong, it confidently fabricates details that are not in any retrieved document.

(a) Which RAGAS metrics would catch this specific failure? State what score you would expect on each and what it means. (b) Is this a retrieval failure or a generation failure? How do you tell the difference? (c) What two architectural changes would reduce the hallucination rate? Be specific: name the component to change and what you change about it. (d) What guardrail at output time would partially mitigate this without changing the retrieval or generation model?

Challenge Problem

Challenge: End-to-End RAG System for an Internal Wiki

Design a complete RAG system for a company's internal knowledge base: 10,000 wiki pages covering HR policies, engineering runbooks, product specs, and customer FAQs. The system must serve 500 concurrent employees with a p99 latency of under 2 seconds. Pages are updated daily.

Your design document must cover all four components:

(1) Indexing pipeline. How do you ingest 10,000 pages? Which chunking strategy and chunk size? Which embedding model (justify your choice of open-source vs. API)? How do you handle daily page updates without re-indexing everything? Which vector database and why?

(2) Retrieval strategy. Dense only, sparse only, or hybrid? If hybrid, how do you weight the two components? Do you add a reranking step? What top-K do you retrieve before and after reranking?

(3) Generation. How do you construct the prompt (system prompt structure, number of chunks injected, max context length)? How do you handle the "lost in the middle" problem? Do you enforce citations?

(4) Evaluation harness. How do you build a test set for a system where ground truth is hard to obtain? Which RAGAS metrics do you track in production? What monitoring alerts do you set up and on which thresholds?

Further Reading

- **RAGAS Paper — Automated Evaluation of RAG** — Es et al., arXiv 2023
  The paper behind the RAGAS library. Defines faithfulness, answer relevancy, context precision, and context recall with formal definitions. Essential reading before building any RAG evaluation harness.
- **Eugene Yan — Patterns for Building LLM-based Systems** — eugeneyan.com, 2023
  Practical patterns for RAG, evals, and guardrails from an ML engineer at Amazon. The RAG and evaluation posts directly map to this session.
- **Weaviate Blog — Vector Database Fundamentals** — weaviate.io/blog
  Best freely available resource on HNSW, IVF, hybrid search, and vector DB internals. Start with "What is a Vector Database?" and "HNSW Explained."
- **Anthropic — Contextual Retrieval** — anthropic.com, 2024
  Anthropic's technique for prepending document context to each chunk before embedding, improving retrieval precision significantly. Free blog post.

Tools and Libraries

- **LangChain** — langchain.com — document loaders, text splitters, retrieval chains
- **LlamaIndex** — llamaindex.ai — alternative to LangChain, strong RAG primitives
- **Chroma** — trychroma.com — open source vector DB, great for prototyping
- **Qdrant** — qdrant.tech — production vector DB with excellent filtering
- **RAGAS** — docs.ragas.io — RAG evaluation framework
- **sentence-transformers** — sbert.net — open source embedding models (E5, BGE)
- **BM25S** — pip install bm25s — fast Python BM25 for sparse retrieval
- **Cohere Rerank** — cohere.com/rerank — managed cross-encoder reranking API

Answers

A1. The two models map text into *different* vector spaces. Cosine similarity between a query vector from `text-embedding-3-small` and a document vector from `text-embedding-ada-002` is meaningless — it is like measuring distance between GPS coordinates and pixel coordinates. Retrieval degrades silently to near-random. Detect it by monitoring the distribution of top-1 cosine similarity scores over time; a sudden drop in average score indicates a space mismatch.

A2. Choose **Qdrant**. Chroma is a development tool, not designed for 50M vectors or complex metadata filtering at production scale. Qdrant is built in Rust, supports HNSW with payload (metadata) filtering at the index level (not post-filter), and is designed for exactly this workload. Pinecone would also be acceptable if the team wants a managed service.

A3. Faithfulness = 0.97 means the LLM is almost always answering using *only* the retrieved context — no hallucination. Context recall = 0.41 means the retrieval step is returning the right information only 41% of the time — it is missing the relevant chunks in 59% of queries. In plain English: the model is honest but uninformed. Fix the retrieval pipeline: review chunking strategy, chunk size, embedding model choice, and consider adding hybrid search.

A4. RRF combines two ranked lists by summing reciprocal ranks: $RRF(d) = \sum_r \frac{1}{k + rank_r(d)}$. It is used instead of score averaging because dense similarity scores (cosine, typically 0.6–0.95) and sparse BM25 scores (can be 0–100+) are on completely different scales. Averaging them directly means one dominates. RRF operates on ranks (ordinal positions), which are scale-invariant and comparable across any retrieval system.